

Sistemas Distribuídos e Computação em Nuvem

Aula 13 — Computação em nuvem, containers e o primeiro deploy

C3 · Nuvem, implantação e segurança · 05/11/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Onde estamos — C3

C1 · Fundamentos (1–7) ✓

C2 · Coordenação e consistência (8–12) ✓

C3 · Nuvem, implantação e segurança (13–18)

Nuvem/containers · orquestração · observabilidade ·
segurança.

 **Aula 13.** Começa o bloco final: tirar o sistema do seu computador e colocá-lo **no ar**.

 Hoje **lança o C3.A2** — o projeto integrador: a **Plataforma de IA como Serviço**, que reúne tudo (microserviços + fila + resiliência) rodando **na nuvem**.

Retomada



Você entregou o C2.A2 (RAG distribuído) com a chamada ao LLM protegida?

- Até aqui, tudo rodava **na sua máquina** (ou na VM). Funciona — mas **ninguém** no mundo acessa.
- Hoje: empacotar o serviço num **container** e publicá-lo na **nuvem** com uma **URL pública**.

Objetivos desta aula

Ao final, você será capaz de:

1. **Distinguir** os modelos de serviço em nuvem: **IaaS**, **PaaS** e **SaaS**.
2. **Explicar elasticidade** e o modelo de **custo por uso**.
3. **Empacotar** um serviço em um **container** (Docker) e **publicá-lo** com URL pública.

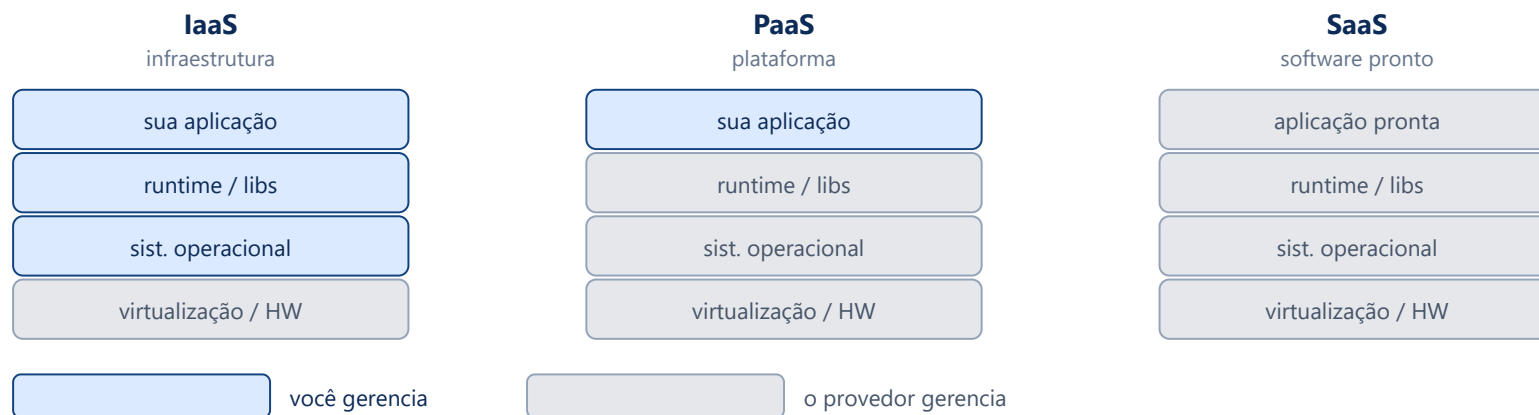
O que é computação em nuvem

Usar, **pela internet** e **sob demanda**, recursos de computação (servidores, armazenamento, banco) de **outra pessoa** — pagando **pelo uso**, sem comprar hardware.

- **Sob demanda:** sobe e desce conforme a necessidade.
- **Pague pelo uso:** não paga servidor ocioso.
- **Autoatendimento:** você provisiona sozinho, em minutos.

 **Em miúdos:** é a diferença entre **comprar um gerador** (e mantê-lo) e **ligar na tomada** e pagar a conta de luz do que usou.

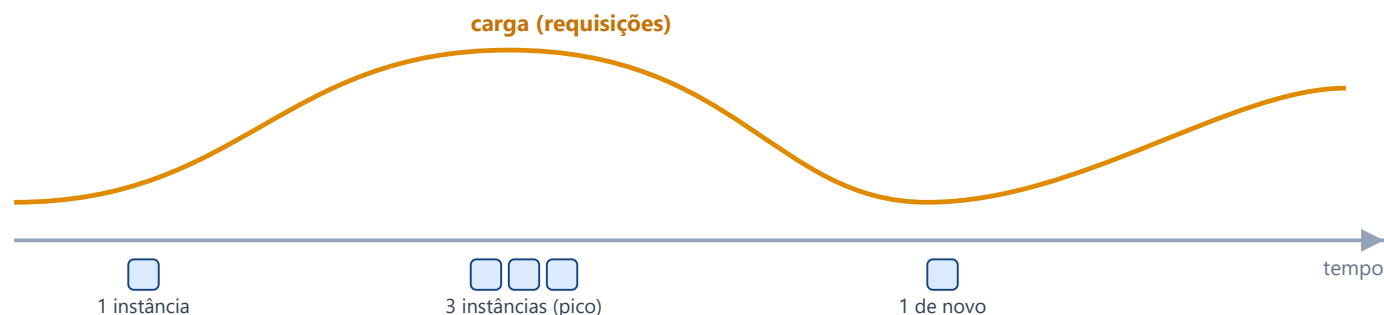
Modelos de serviço: IaaS, PaaS, SaaS



💡 **Em miúdos (pizza):** **IaaS** = massa pronta, você monta e assa; **PaaS** = pizzaria assa, você só escolhe o recheio (seu código); **SaaS** = pizza entregue pronta (ex.: Gmail).

Elasticidade e custo

- **Elasticidade:** a nuvem **adiciona** instâncias quando a carga sobe e **remove** quando cai — **automaticamente**.
- **Custo por uso:** você paga pelas instâncias **enquanto existem**. Escalar para zero = **custo zero**.



💡 **Em miúdos:** é contratar **mais caixas no supermercado** quando a fila cresce e **dispensá-los** quando esvazia — você paga só pelas horas trabalhadas.

O problema do "na minha máquina funciona"

O serviço depende de uma **versão** de Python, de **bibliotecas**, de variáveis de ambiente... Mudou a máquina, **quebrou**. O **container** resolve: empacota **o app + o ambiente inteiro** numa unidade que roda **igual** em qualquer lugar.



💡 Container **compartilha** o S.O. do host → sobe em **segundos** e pesa **MB** (a VM leva minutos e pesa GB). Por isso é a unidade de deploy na nuvem.

Imagem × container · o Dockerfile

- **Imagem:** o **pacote** (receita assada) — app + dependências + instruções de execução. Imutável.
- **Container:** uma **instância em execução** da imagem. Você sobe **vários** da mesma imagem.

```
FROM python:3.12-slim          # base: S.O. mínimo + Python
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt # instala as deps (vira uma camada em cache)
COPY app.py .
EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

💡 **Em miúdos:** a **imagem** é a receita assada; o **container** é cada fatia servida. `0.0.0.0` = "ouça em todas as interfaces", senão ninguém de fora acessa.

Laboratório

Containerizar o serviço e publicar na nuvem

Mão na massa — siga o Guia do Laboratório

Hoje são **4 missões**: rodar o serviço local, **containerizar** (build + run), **publicar** na nuvem com URL pública e **lançar o C3.A2** (containerizar os serviços do projeto).

👉 **Abrir o Guia do Laboratório »**

no índice do site: Aula 13 → [Guia do Lab](#)

💡 Exemplo pronto e testado: `exemplos/aula13/` — `app.py` (FastAPI que mostra o host), `Dockerfile`, `requirements.txt` e `.dockerignore`.

O que o exemplo mostra (aquecimento)

```
# 1) local (sem Docker): sobe e testa
uvicorn app:app --port 8000      # GET / -> {"mensagem":"Ola da nuvem!","host":"...","versao":"1.0"}

# 2) containeriza e roda igual, isolado:
docker build -t servico-nuvem .
docker run -p 8000:8000 servico-nuvem  # o "host" no JSON agora e o ID do container
```

💡 O campo `host` muda quando roda em container (vira o ID do container) — é como você vai **ver** réplicas diferentes respondendo quando escalar.

No seu trabalho — C3.A2 (lançado hoje)

O **C3.A2** é o **projeto integrador**: API Gateway + serviço de inferência + worker de fila + armazenamento vetorial, **containerizados** e **na nuvem**.

- **Hoje**: cada serviço ganha o **seu** `Dockerfile` e **builda** individualmente.
- Na **próxima aula**: subir **tudo junto** com um comando (orquestração).

💡 Puxa para o kit: comece pelo serviço mais simples. Um `Dockerfile` por serviço, cada um buildando sozinho, é a meta de hoje.

Atividade para casa

1. **Publique** o serviço de exemplo (ou um do seu projeto) em uma nuvem **free tier** (Render/Railway) e obtenha uma **URL pública**.
2. **Containerize todos** os serviços do **C3.A2** (um `Dockerfile` por serviço, cada um buildando).
3. **Registre** no README o comando de build/run de cada serviço.

📌 **Entregar até a próxima aula:** a **URL pública** do serviço no ar + todos os serviços do C3.A2 containerizados. Roteiro completo no **Guia do Laboratório**.

◆ Foco ENADE

O que costuma cair:

- Modelos de serviço **IaaS × PaaS × SaaS** (quem gerencia o quê).
- **Elasticidade** e **custo por uso** (escala sob demanda).
- **Container × VM** (compartilha o S.O. do host; leve e rápido).
- **Portabilidade**: "roda igual em qualquer lugar".

Termos-chave: Nuvem · IaaS/PaaS/SaaS · Elasticidade · Container · Imagem · Dockerfile · Deploy

💡 Pegadinha: confundir **container** com **VM**. Container compartilha o S.O. do host; VM carrega um S.O. convidado inteiro.

Questão de autoavaliação (estilo ENADE)

Uma equipe quer **implantar sua própria aplicação** sem gerenciar sistema operacional nem servidores, enviando **apenas o código** para uma plataforma que cuida do runtime e da execução. O modelo de serviço em nuvem mais adequado é:

- A) IaaS, pois oferece máquinas virtuais cruas
- B) **PaaS, pois entrega a plataforma e o runtime, bastando enviar o código**
- C) SaaS, pois entrega um software pronto ao usuário final
- D) nenhum, pois exige comprar servidores físicos
- E) on-premises, pois roda no datacenter próprio

Resolução — alternativa B

- **PaaS**: o provedor cuida de S.O., runtime e escala; a equipe entrega **só o código** (ex.: Render, Railway, App Engine).
- **IaaS** (A) daria a VM crua (você gerencia o S.O.); **SaaS** (C) é software pronto para o usuário final, não para hospedar o seu app.

💡 "Enviar só o código" é a assinatura do **PaaS** — exatamente o free tier que você usa no deploy.

Fora da sala · Glossário

Para estudar

- **Coulouris**, cap. 7 — Virtualização e nuvem (visão geral).
- Docs do **Docker** (Get Started) e do **Render/Railway**.
- Rode o exemplo: compare o `host` local × no container.

Glossário

- **IaaS/PaaS/SaaS**: camadas de serviço (infra / plataforma / software).
- **Elasticidade**: ajustar instâncias conforme a carga.
- **Container**: app + ambiente empacotados; compartilha o S.O. do host.
- **Imagem**: o pacote imutável; **container** é a imagem em execução.
- **Dockerfile**: receita que descreve a imagem.

Até a próxima aula 🚀

Publique o serviço (URL pública) e containerize os serviços do C3.A2.

Próxima (Aula 14): Orquestração, serverless e computação na borda — subir a stack inteira com um comando e entregar funções sem gerenciar servidor.

← Anterior

Aula 12 · Revisão C2

≡ Índice

todas as aulas